AI Assistant Coding

Assignment-15

Name : Shahalam

Hall Ticket No. : 2303A510C1

Batch No. : 06

Task 1– Real-Time Application: Online Food Ordering API

Scenario:

Design a simple API for an online food ordering system.

Requirements:

• Endpoints required:

• GET /menu → List available dishes

• POST /order → Place a new order

• GET /order/{id} → Track order status

• PUT /order/{id} → Update an existing order (e.g., change items)

• DELETE /order/{id} → Cancel an order

• **AI should generate:**

• REST API code

• Suggested improvements (like authentication, pagination) **Expected Output:**

• Fully working API simulating a food ordering backend.

Prompt:

Create a Node.js Express REST API for an online food ordering system.

Requirements:

- GET /menu → return list of dishes

- POST /order → create new order with items

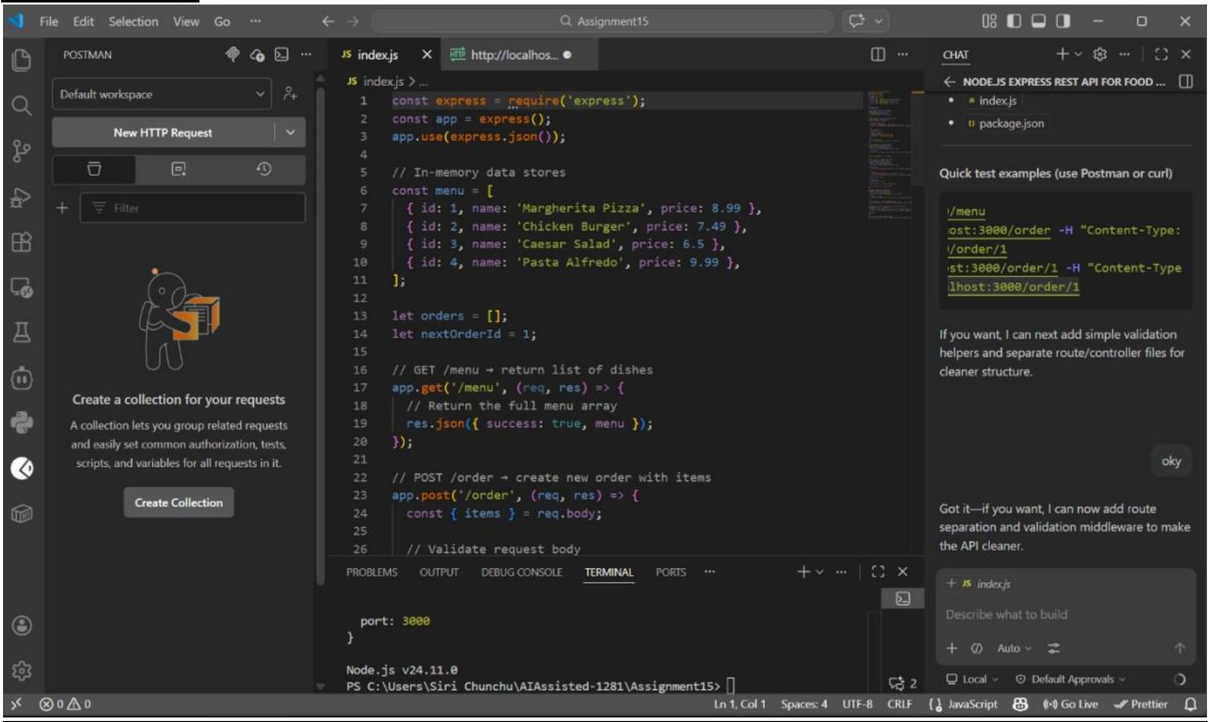
- GET /order/:id → get order status

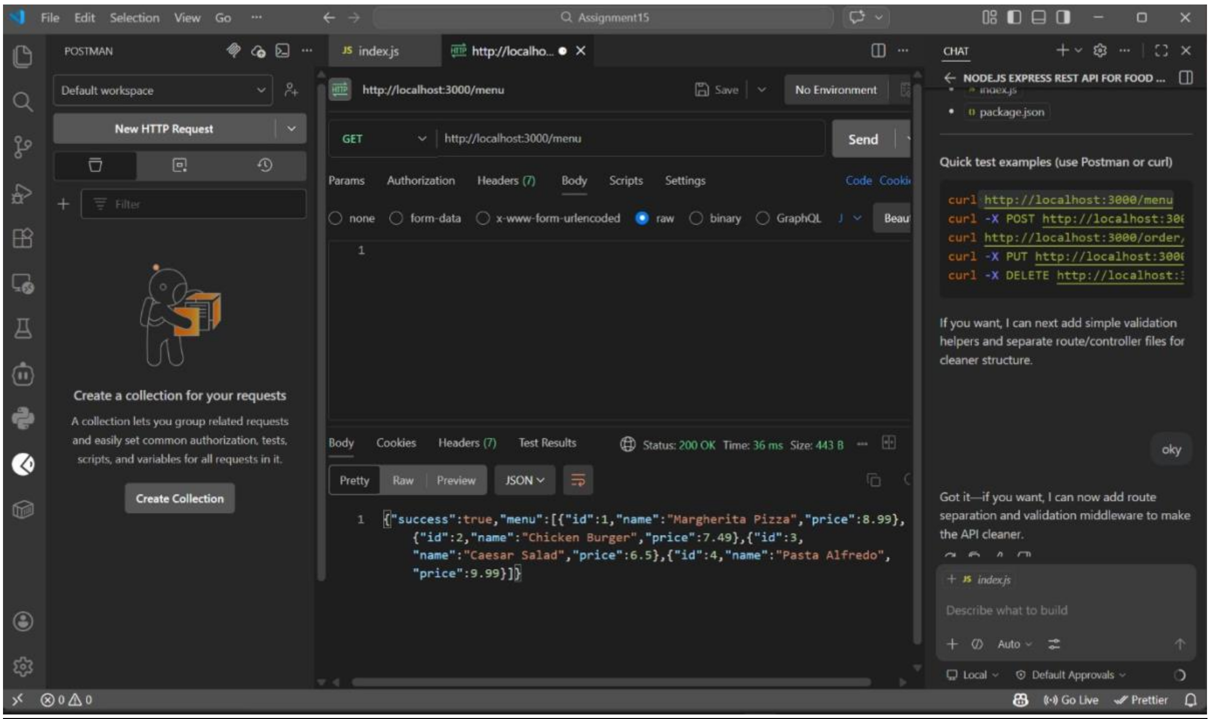
- PUT /order/:id → update order items

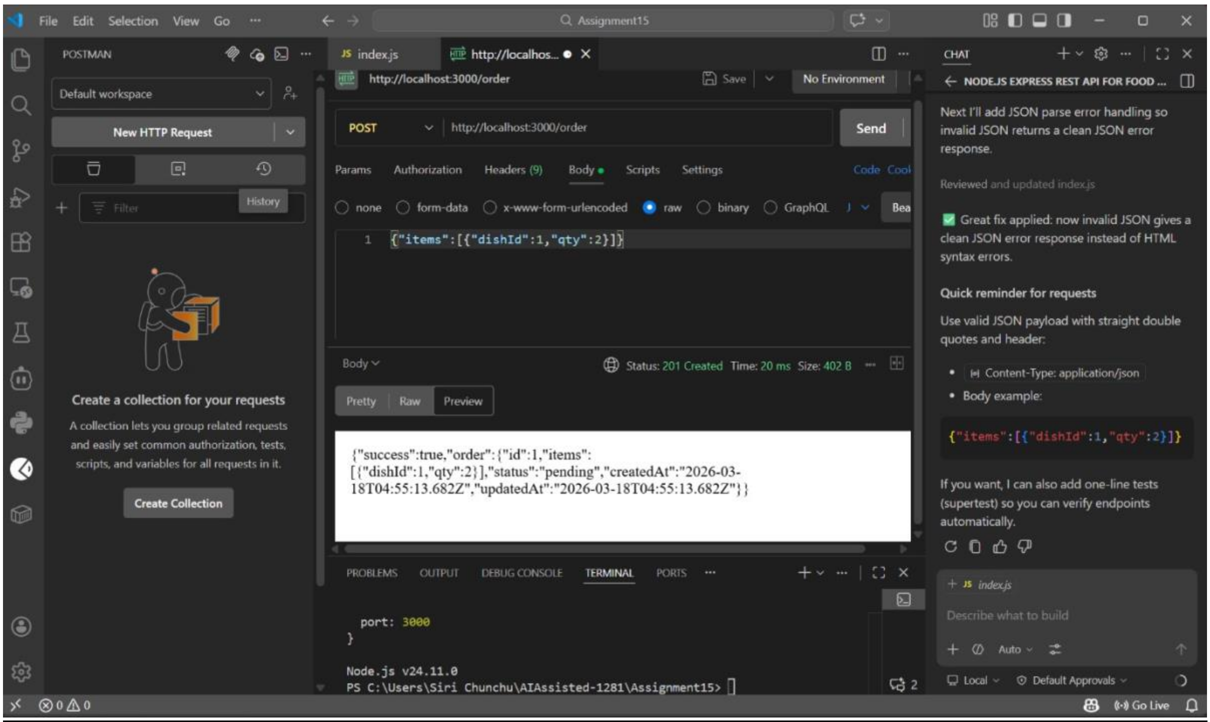
- DELETE /order/:id → cancel order

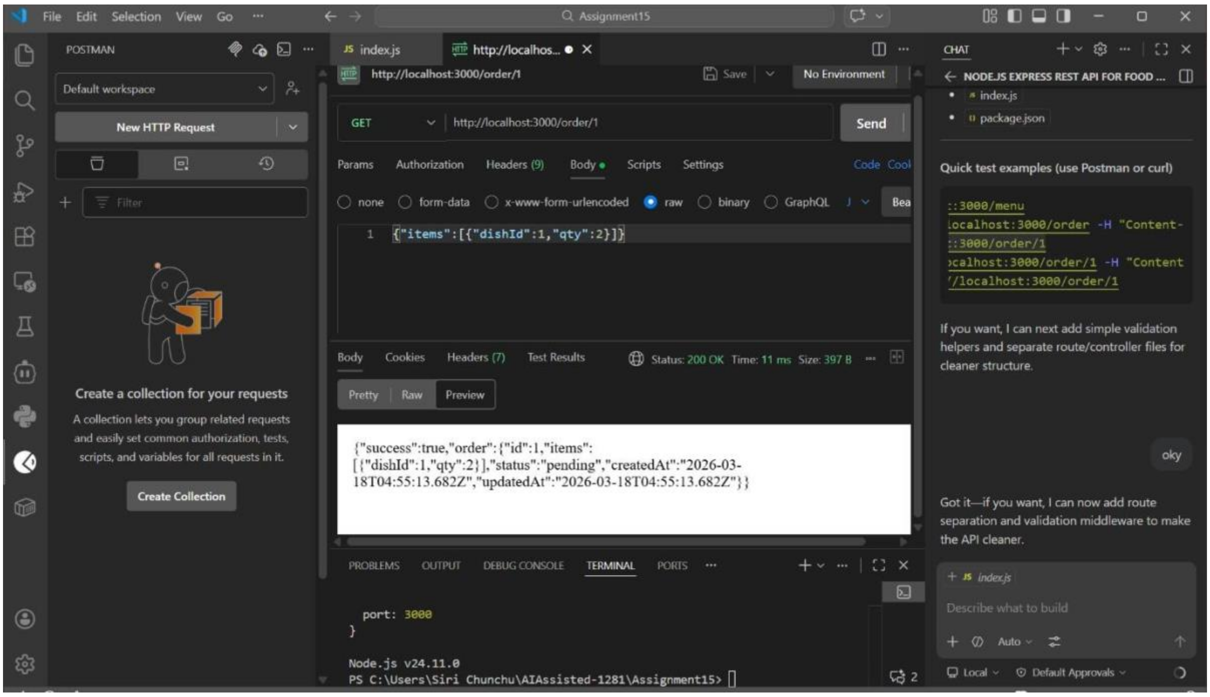
Use in-memory arrays. Add proper comments for each endpoint.

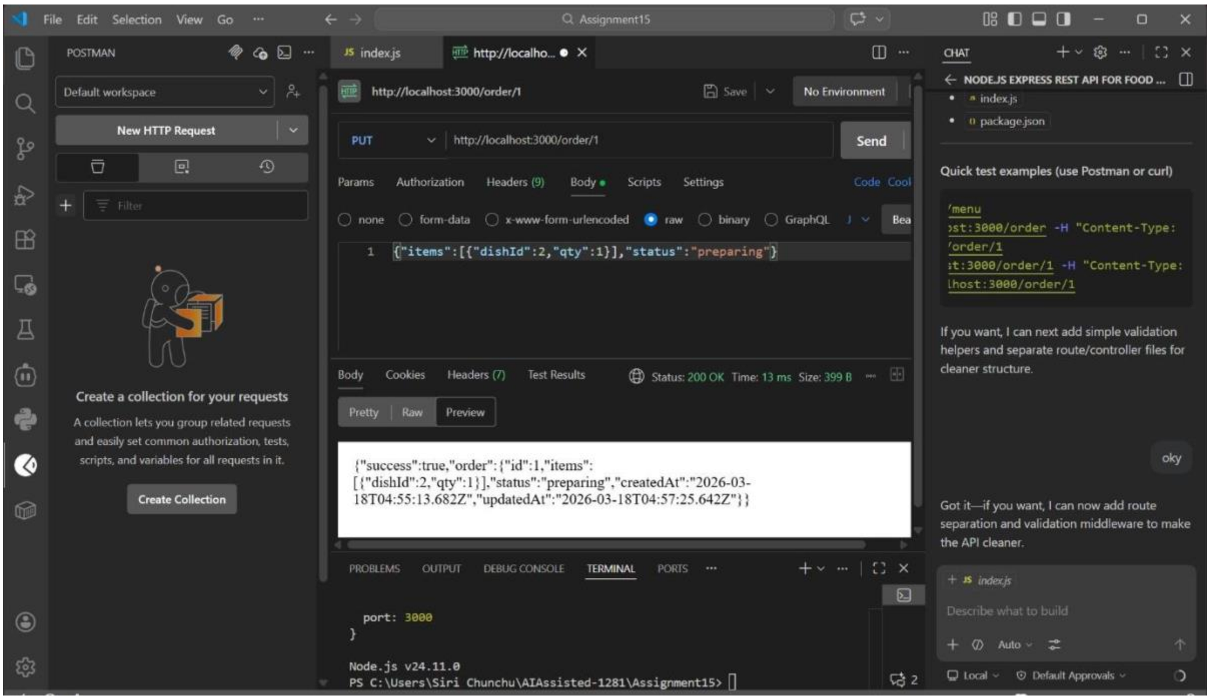
AI Generated:

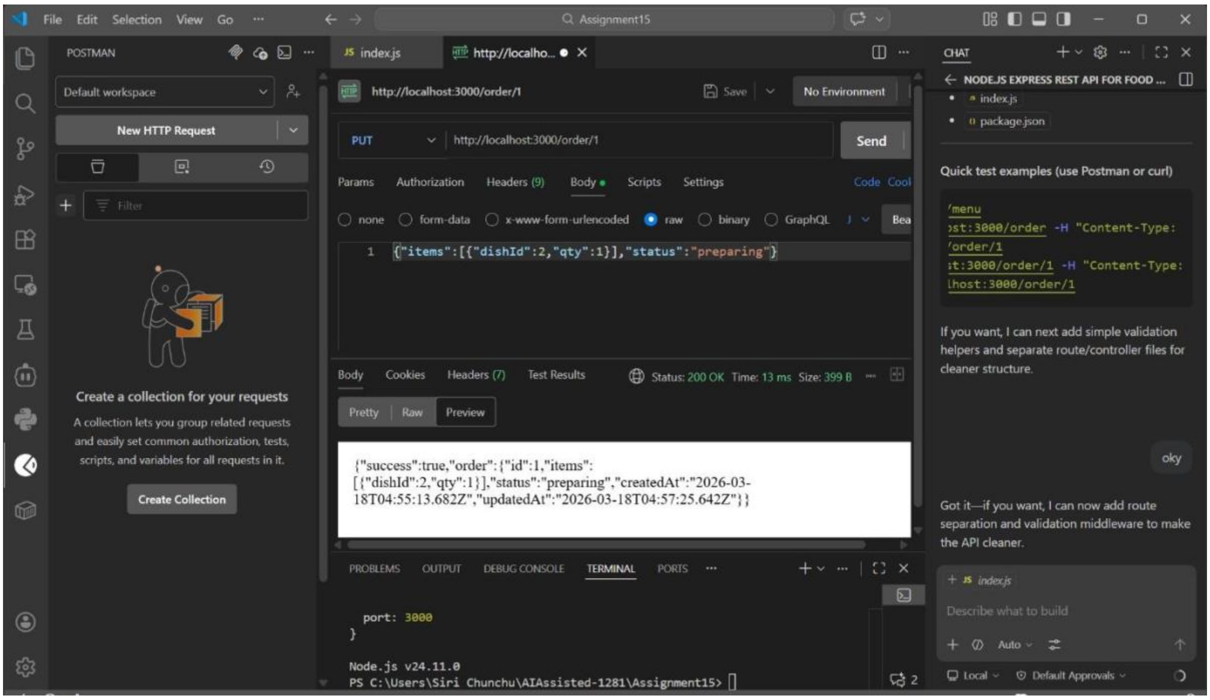












Explanation

This API simulates a basic online food ordering system using REST principles.
The **GET /menu** endpoint retrieves available dishes.
The **POST /order** endpoint allows users to place a new order by sending selected items.
The **GET /order/:id** endpoint helps in tracking the status of an order.
The **PUT /order/:id** endpoint is used to update order details such as items.
The **DELETE /order/:id** endpoint cancels an existing order.
All data is stored in an in-memory array, making it simple and fast for testing.

Task 2 – Employee Payroll API

Task:

Create an API for managing employees and their salaries.

Instructions:

• **Endpoints required:**

• GET /employees → List all employees

• POST /employees → Add a new employee with salary details

• PUT /employees/{id}/salary → Update salary of an employee • DELETE /employees/{id} → Remove employee from system

• **Use AI to:**

• Suggest data model structure.

• Add comments/docstrings for all endpoints.

Expected Output:

• Payroll management API with validation

• Secure and structured data handling

• Clear API documentation

Prompt:

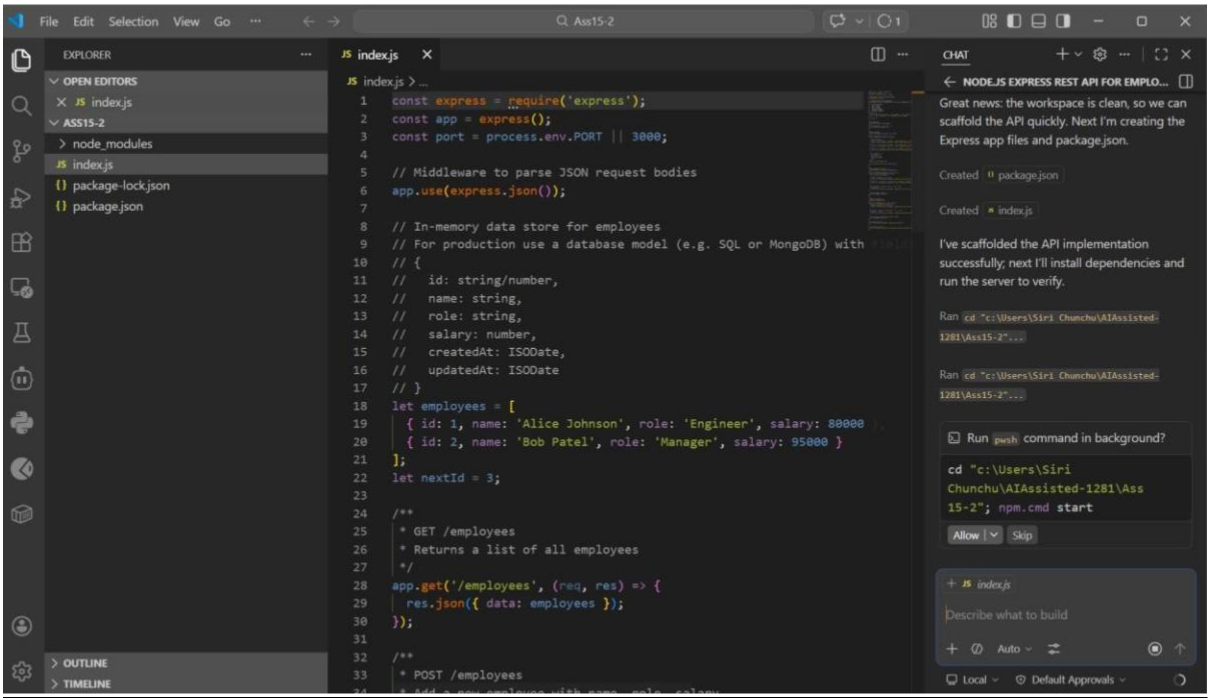
Create a Node.js Express REST API to manage employees and salaries.

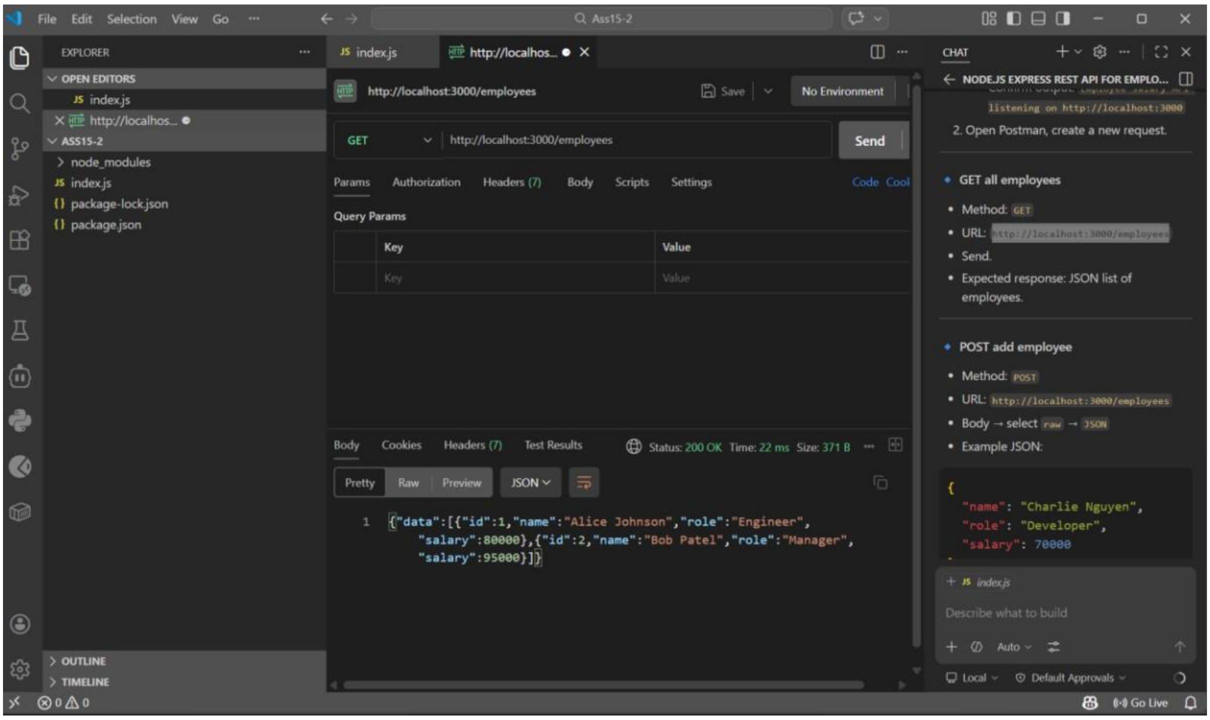
Requirements:

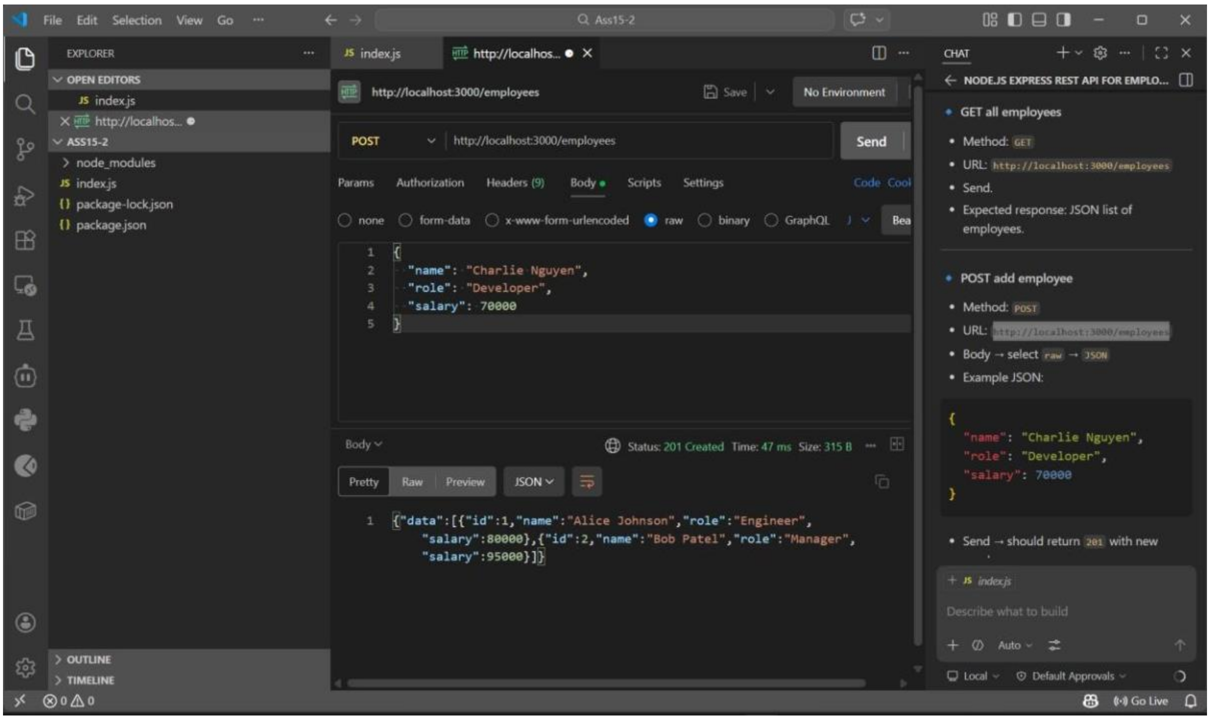
- GET /employees → list all employees
- POST /employees → add employee with name, role, salary
- PUT /employees/:id/salary → update salary
- DELETE /employees/:id → remove employee Add validation and comments for each endpoint.

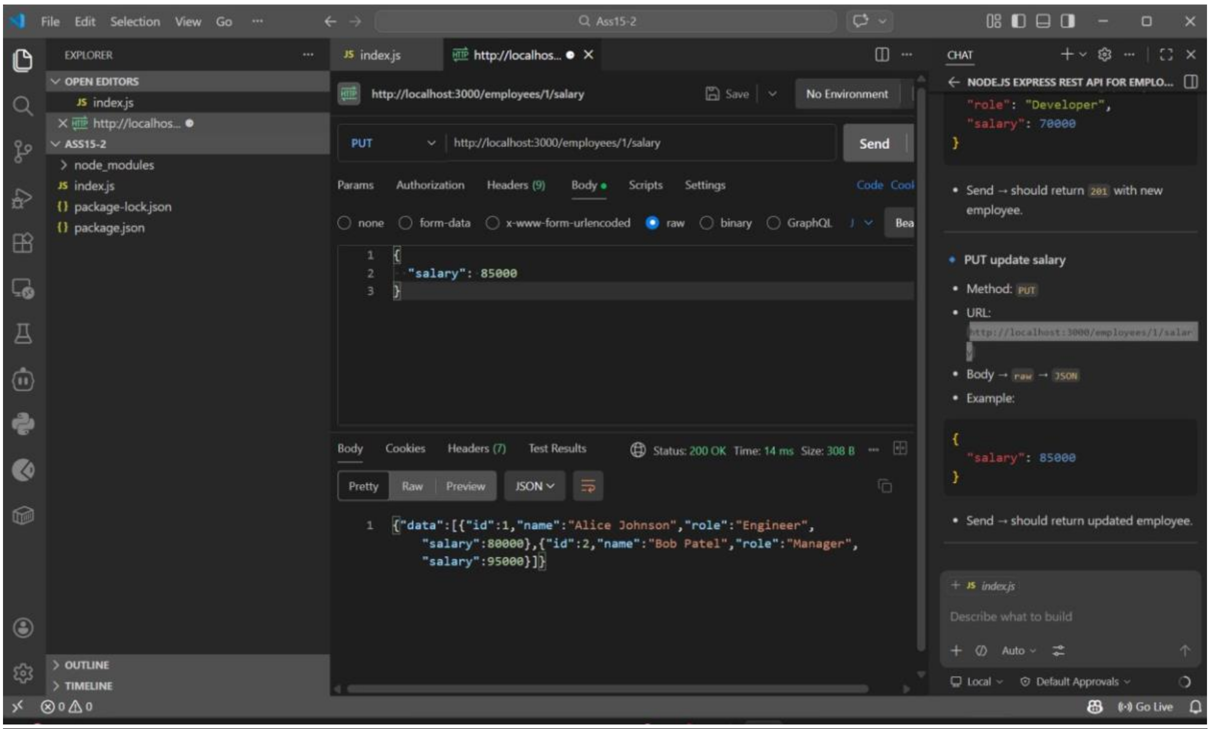
Also suggest a proper data model.

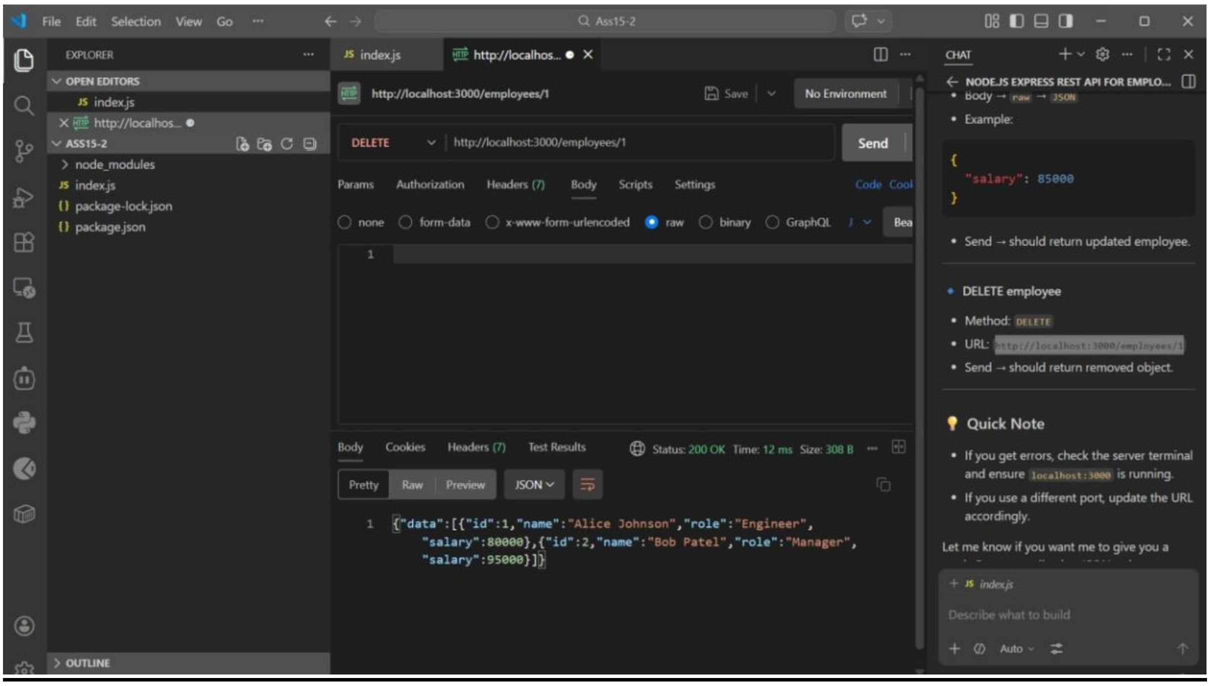
AI Generated:


File Edit Selection View Go
Ass15-2
EXPLORER
OPEN EDITORS
index.js
ASS15-2
node_modules
index.js
package-lock.json
package.json
OUTLINE
TIMELINE
index.js
const express = require('express');
const app = express();
const port = process.env.PORT || 3000;
// Middleware to parse JSON request bodies
app.use(express.json());
// In-memory data store for employees
// For production use a database model (e.g. SQL or MongoDB) with
// {
//   id: string/number,
//   name: string,
//   role: string,
//   salary: number,
//   createdAt: ISODate,
//   updatedAt: ISODate
// }
let employees = [
{ id: 1, name: 'Alice Johnson', role: 'Engineer', salary: 80000
{ id: 2, name: 'Bob Patel', role: 'Manager', salary: 95000 }
];
let nextId = 3;
/**
* GET /employees
* Returns a list of all employees
*/
app.get('/employees', (req, res) => {
res.json({ data: employees });
});
/**
* POST /employees
CHAT
NODE.JS EXPRESS REST API FOR EMPLO...
Great news: the workspace is clean, so we can scaffold the API quickly. Next I'm creating the Express app files and package.json.
Created package.json
Created index.js
I've scaffolded the API implementation successfully; next I'll install dependencies and run the server to verify.
Ran cd "c:\Users\Siri Chunchu\AIAssisted-1281\Ass15-2"...
Ran cd "c:\Users\Siri Chunchu\AIAssisted-1281\Ass15-2"...
Run pwsh command in background?
cd "c:\Users\Siri Chunchu\AIAssisted-1281\Ass 15-2"; npm.cmd start
Allow
Skip
index.js
Describe what to build
Auto
Local
Default Approvals


File Edit Selection View Go
Ass15-2
EXPLORER
OPEN EDITORS
index.js
http://localhos...
ASS15-2
node_modules
index.js
package-lock.json
package.json
OUTLINE
TIMELINE
index.js
http://localhos...
http://localhost:3000/employees
Save
No Environment
GET
http://localhost:3000/employees
Send
Params Authorization Headers (7) Body Scripts Settings
Code Cool
Query Params
Key
Value
Body Cookies Headers (7) Test Results
Status: 200 OK Time: 22 ms Size: 371 B
Pretty Raw Preview JSON
{"data":[{"id":1,"name":"Alice Johnson","role":"Engineer","salary":80000},{"id":2,"name":"Bob Patel","role":"Manager","salary":95000}]}
CHAT
NODE.JS EXPRESS REST API FOR EMPLO...
listening on http://localhost:3000
2. Open Postman, create a new request.
GET all employees
Method: GET
URL: http://localhost:3000/employees
Send.
Expected response: JSON list of employees.
POST add employee
Method: POST
URL: http://localhost:3000/employees
Body → select raw → JSON
Example JSON:
{
"name": "Charlie Nguyen",
"role": "Developer",
"salary": 70000
index.js
Describe what to build
Auto
Local
Default Approvals
Go Live


File Edit Selection View Go
Ass15-2
EXPLORER
OPEN EDITORS
index.js
http://localhos...
ASS15-2
node_modules
index.js
package-lock.json
package.json
OUTLINE
TIMELINE
http://localhost:3000/employees
Save
No Environment
POST
http://localhost:3000/employees
Send
Params Authorization Headers (9) Body Scripts Settings
none form-data x-www-form-urlencoded raw binary GraphQL
{
"name": "Charlie Nguyen",
"role": "Developer",
"salary": 70000
}
Body
Status: 201 Created Time: 47 ms Size: 315 B
Pretty Raw Preview JSON
{"data":[{"id":1,"name":"Alice Johnson","role":"Engineer",
"salary":80000},{"id":2,"name":"Bob Patel","role":"Manager",
"salary":95000}]}
CHAT
NODE.JS EXPRESS REST API FOR EMPLO...
GET all employees
Method: GET
URL: http://localhost:3000/employees
Send.
Expected response: JSON list of employees.
POST add employee
Method: POST
URL: http://localhost:3000/employees
Body → select raw → JSON
Example JSON:
{
"name": "Charlie Nguyen",
"role": "Developer",
"salary": 70000
}
Send → should return 201 with new
index.js
Describe what to build
Auto
Local
Default Approvals
Go Live


File Edit Selection View Go
Ass15-2
EXPLORER
OPEN EDITORS
index.js
http://localhos...
ASS15-2
node_modules
index.js
package-lock.json
package.json
OUTLINE
TIMELINE
http://localhost:3000/employees/1/salary
Save
No Environment
PUT
http://localhost:3000/employees/1/salary
Send
Params Authorization Headers (9) Body Scripts Settings
none form-data x-www-form-urlencoded raw binary GraphQL
{
"salary": 85000
}
Body Cookies Headers (7) Test Results
Status: 200 OK Time: 14 ms Size: 308 B
Pretty Raw Preview JSON
{"data":[{"id":1,"name":"Alice Johnson","role":"Engineer",
"salary":80000},{"id":2,"name":"Bob Patel","role":"Manager",
"salary":95000}]}
CHAT
NODE.JS EXPRESS REST API FOR EMPLO...
"role": "Developer",
"salary": 70000
}
Send → should return 201 with new employee.
PUT update salary
Method: PUT
URL:
http://localhost:3000/employees/1/salary
Body → raw → JSON
Example:
{
"salary": 85000
}
Send → should return updated employee.
index.js
Describe what to build
Auto
Local
Default Approvals
Go Live



Explanation

This API manages employee records and salary details.
The **GET /employees** endpoint retrieves all employee data.
The **POST /employees** endpoint adds a new employee with salary information.

The **PUT /employees/:id/salary** endpoint updates the salary of a specific employee.
The **DELETE /employees/:id** endpoint removes an employee from the system.
Basic validation ensures correct data input before processing.

Task 3: Student Information REST API

Scenario:

A university wants to digitize student records for easy retrieval. You need to build a

Student Management API that allows:

• Adding a new student (name, roll number, course, year)

• Fetching details of all students

• Updating student information

• Deleting a student record

Implementation Instructions:

1. Use Node.js + Express (or Python Flask/Django REST) with GitHub

Copilot to auto-generate routes.

2. Store student data in an in-memory array or SQLite/PostgreSQL

database.

Test API endpoints with Postman.

Prompt:

Create a Student Management REST API using Node.js and Express.

Features:

- Add student (name, roll number, course, year)

- Get all students

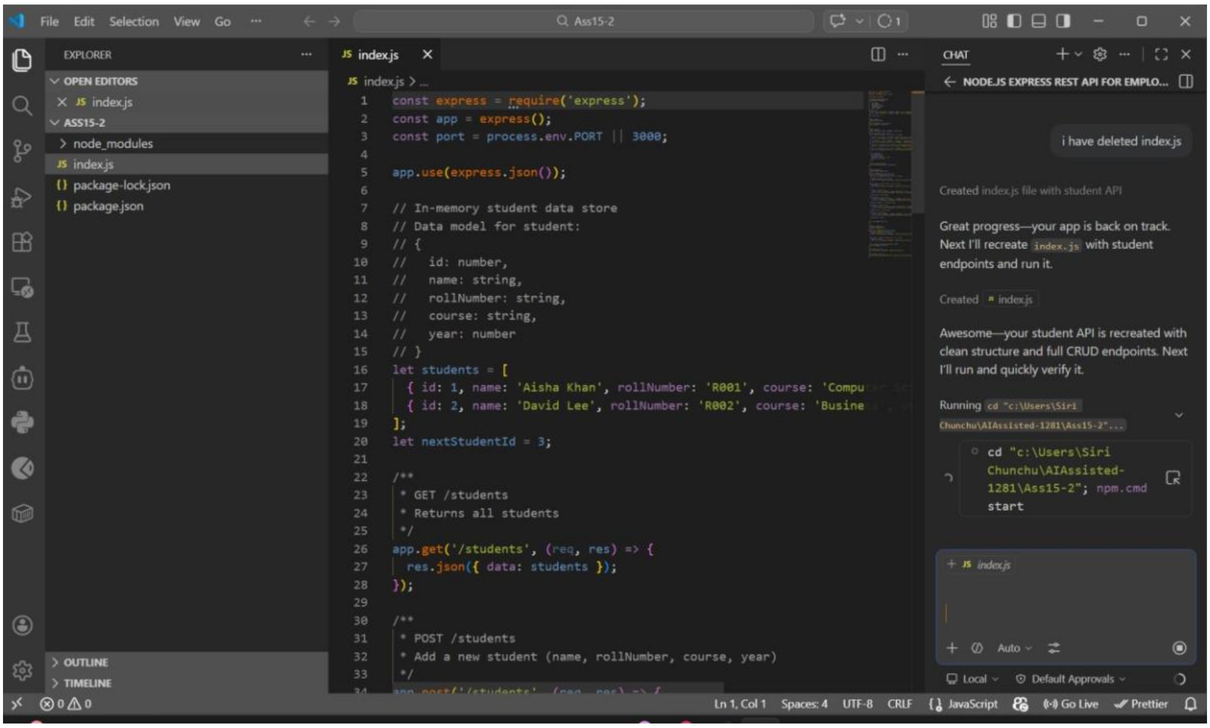
- Update student details

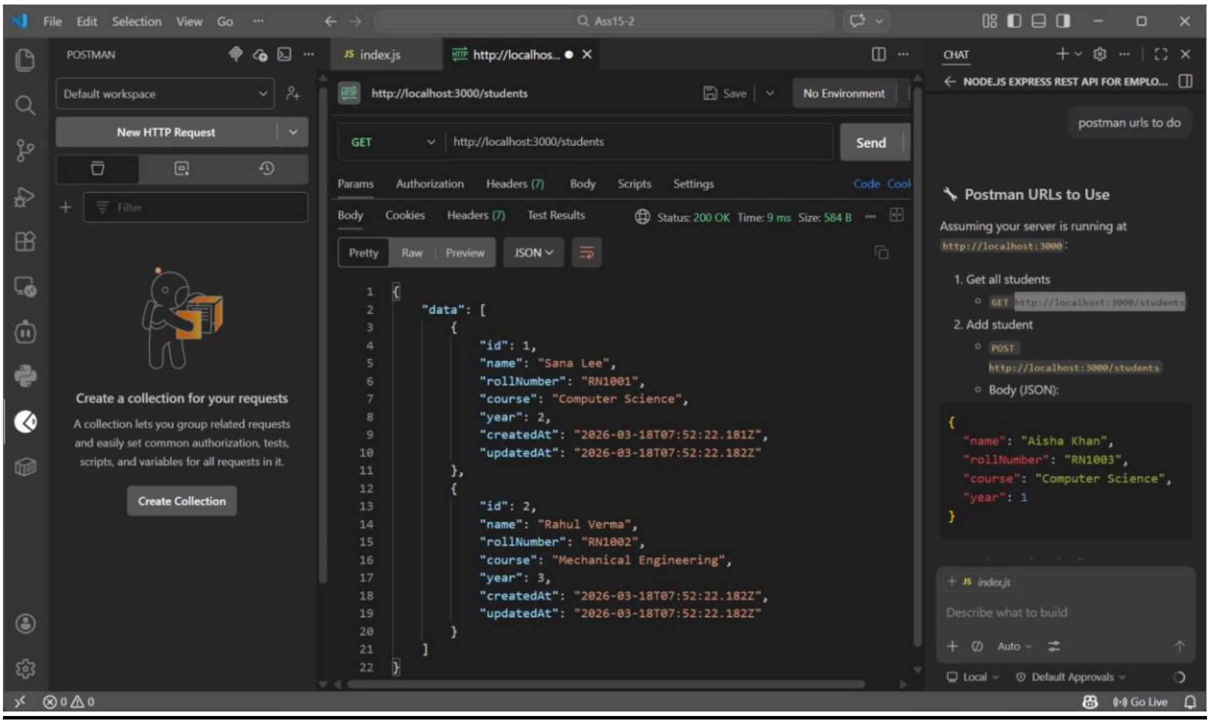
- Delete student

Store data in an in-memory array.

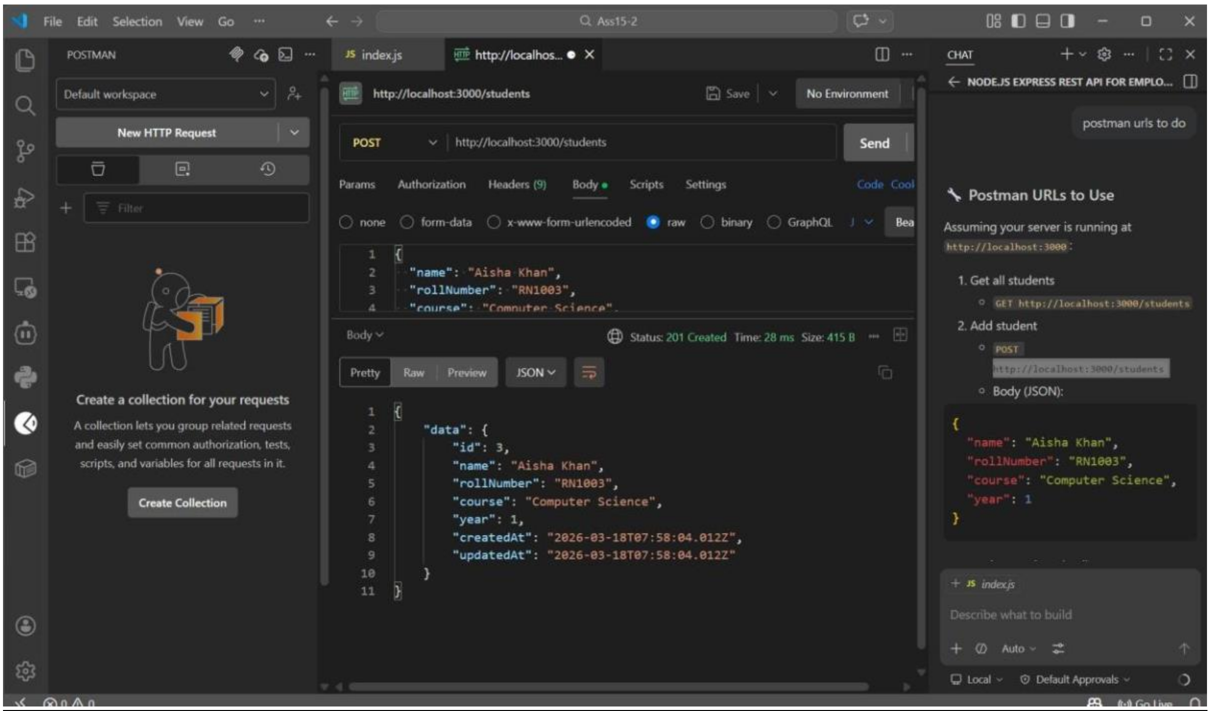
Add comments and clean structure.

AI Generated:

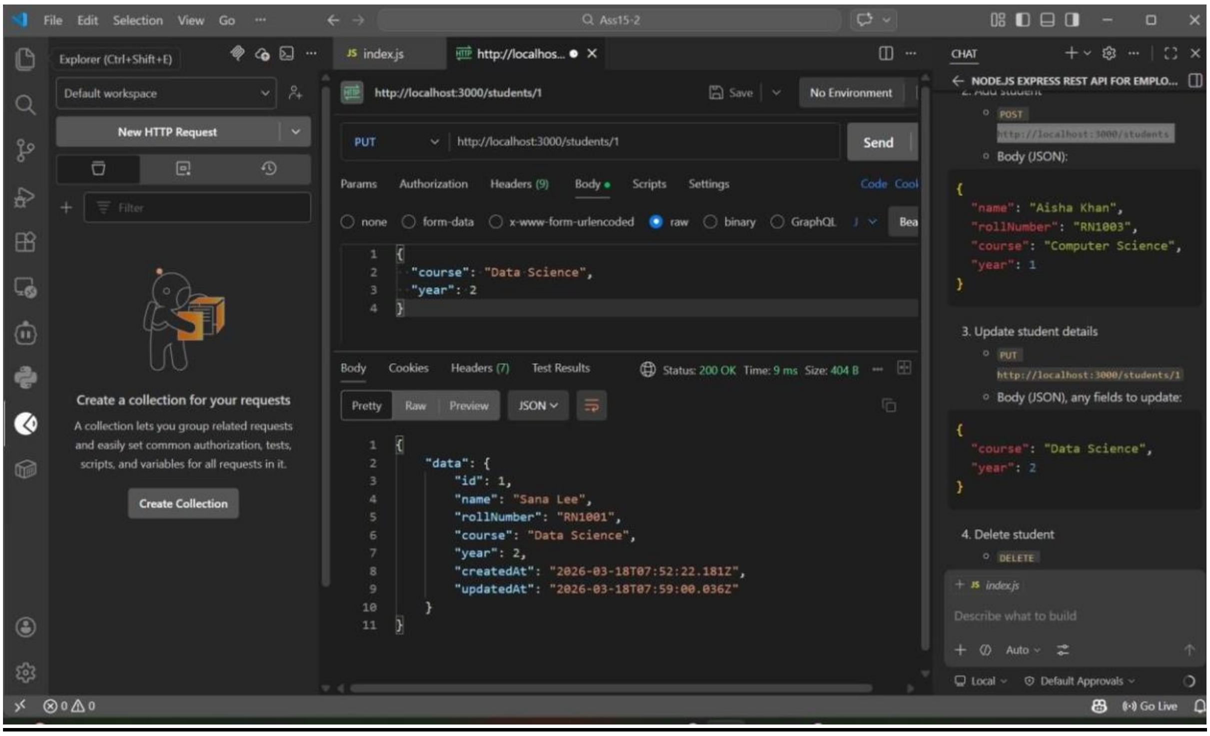


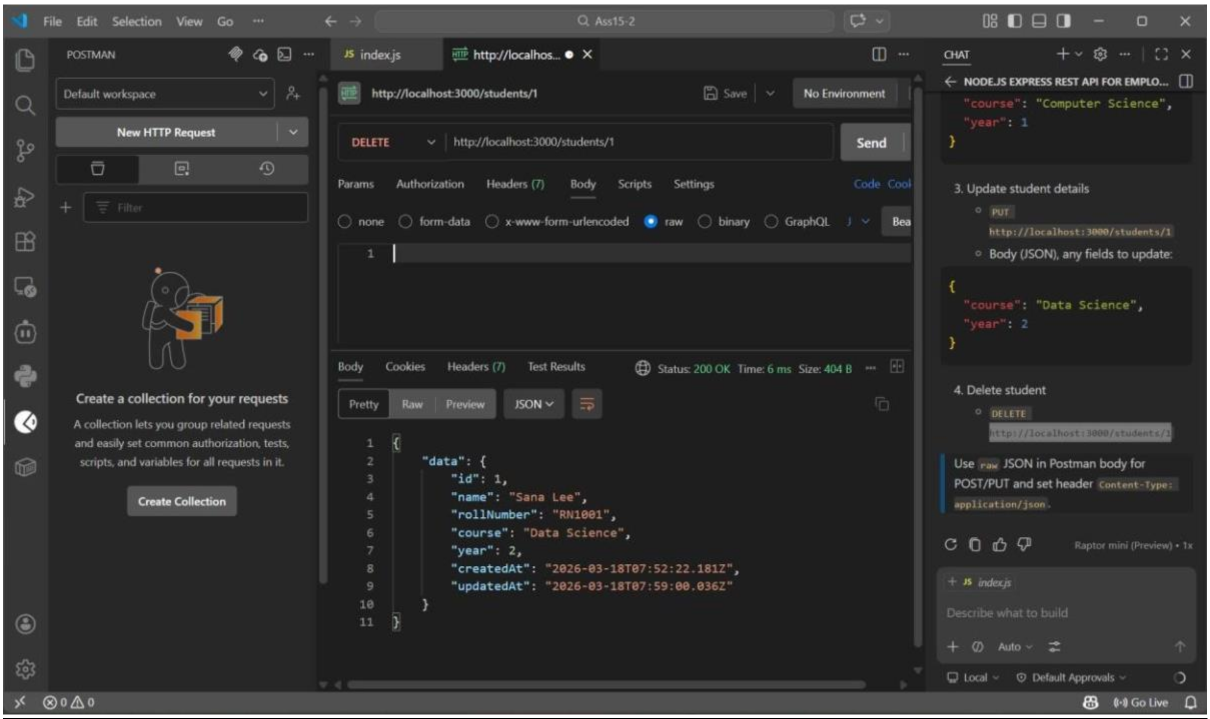


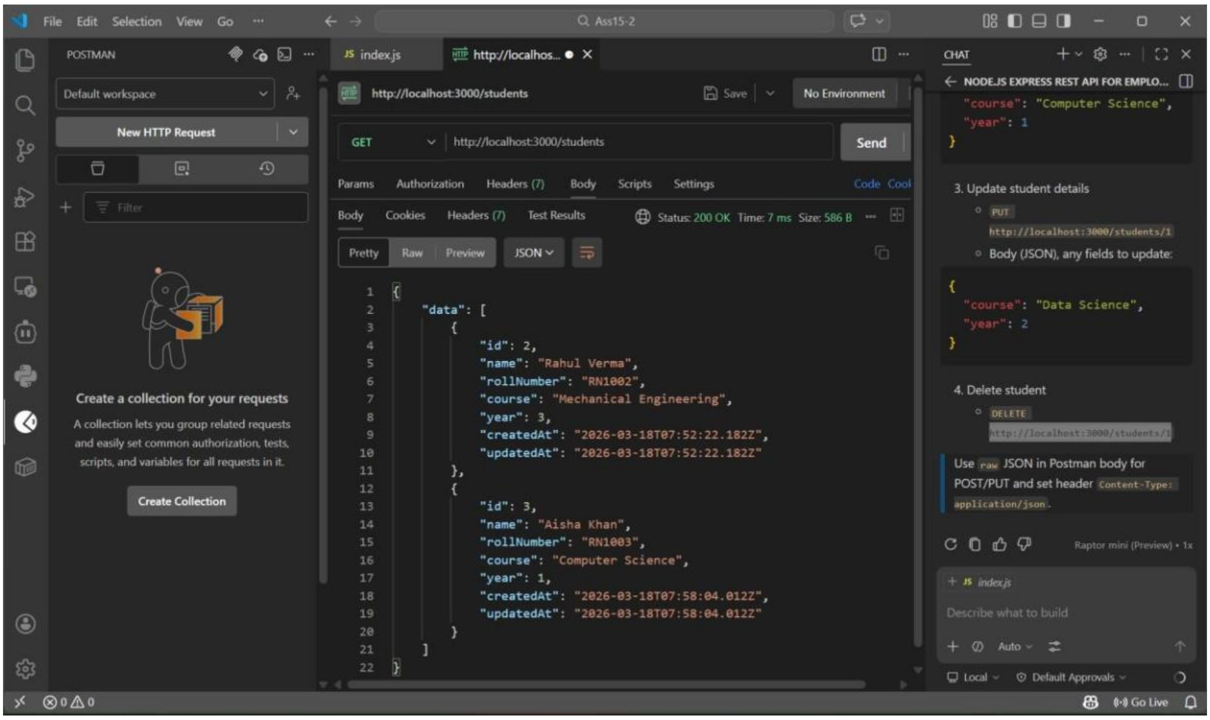
File Edit Selection View Go
Ass15-2
POSTMAN
index.js
http://localhos...
Default workspace
New HTTP Request
Filter
Create a collection for your requests
A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.
Create Collection
http://localhost:3000/students
Save
No Environment
GET
http://localhost:3000/students
Send
Params Authorization Headers (7) Body Scripts Settings
Code Cookies
Body Cookies Headers (7) Test Results
Status: 200 OK Time: 9 ms Size: 584 B
Pretty Raw Preview JSON
{
"data": [
{
"id": 1,
"name": "Sana Lee",
"rollNumber": "RN1001",
"course": "Computer Science",
"year": 2,
"createdAt": "2026-03-18T07:52:22.181Z",
"updatedAt": "2026-03-18T07:52:22.182Z"
},
{
"id": 2,
"name": "Rahul Verma",
"rollNumber": "RN1002",
"course": "Mechanical Engineering",
"year": 3,
"createdAt": "2026-03-18T07:52:22.182Z",
"updatedAt": "2026-03-18T07:52:22.182Z"
}
]
}
CHAT
NODE.JS EXPRESS REST API FOR EMPLO...
postman urls to do
Postman URLs to Use
Assuming your server is running at http://localhost:3000:
1. Get all students
GET http://localhost:3000/students
2. Add student
POST
http://localhost:3000/students
Body (JSON):
{
"name": "Aisha Khan",
"rollNumber": "RN1003",
"course": "Computer Science",
"year": 1
}
index.js
Describe what to build
Auto
Local
Default Approvals
Go Live



File Edit Selection View Go
Ass15-2
POSTMAN
index.js
http://localhos...
Default workspace
New HTTP Request
Filter
Create a collection for your requests
A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.
Create Collection
http://localhost:3000/students
Save
No Environment
POST
http://localhost:3000/students
Send
Params Authorization Headers (9) Body Scripts Settings
Code Cookies
none form-data x-www-form-urlencoded raw binary GraphQL
{
"name": "Aisha Khan",
"rollNumber": "RN1003",
Body
Status: 201 Created Time: 28 ms Size: 415 B
Pretty Raw Preview JSON
{
"data": {
"id": 3,
"name": "Aisha Khan",
"rollNumber": "RN1003",
"course": "Computer Science",
"year": 1,
"createdAt": "2026-03-18T07:58:04.012Z",
"updatedAt": "2026-03-18T07:58:04.012Z"
}
}
CHAT
NODE.JS EXPRESS REST API FOR EMPLO...
postman urls to do
Postman URLs to Use
Assuming your server is running at http://localhost:3000:
1. Get all students
GET http://localhost:3000/students
2. Add student
POST
http://localhost:3000/students
Body (JSON):
{
"name": "Aisha Khan",
"rollNumber": "RN1003",
"course": "Computer Science",
"year": 1
}
index.js
Describe what to build
Auto
Local
Default Approvals


http://localhost:3000/students/1
PUT
http://localhost:3000/students/1
Send
Create a collection for your requests
A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.
Create Collection
{
"course": "Data Science",
"year": 2
}
Status: 200 OK Time: 9 ms Size: 404 B
{
"data": {
"id": 1,
"name": "Sana Lee",
"rollNumber": "RN1001",
"course": "Data Science",
"year": 2,
"createdAt": "2026-03-18T07:52:22.181Z",
"updatedAt": "2026-03-18T07:59:00.036Z"
}
}
NODE.JS EXPRESS REST API FOR EMPLO...
POST
Body (JSON):
{
"name": "Aisha Khan",
"rollNumber": "RN1003",
"course": "Computer Science",
"year": 1
}
3. Update student details
PUT
http://localhost:3000/students/1
Body (JSON), any fields to update:
{
"course": "Data Science",
"year": 2
}
4. Delete student
DELETE


POSTMAN
http://localhost:3000/students/1
DELETE
http://localhost:3000/students/1
Send
Create a collection for your requests
A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.
Create Collection
Status: 200 OK Time: 6 ms Size: 404 B
{
"data": {
"id": 1,
"name": "Sana Lee",
"rollNumber": "RN1001",
"course": "Data Science",
"year": 2,
"createdAt": "2026-03-18T07:52:22.181Z",
"updatedAt": "2026-03-18T07:59:00.036Z"
}
}
NODE.JS EXPRESS REST API FOR EMPLO...
3. Update student details
PUT
http://localhost:3000/students/1
Body (JSON), any fields to update:
{
"course": "Data Science",
"year": 2
}
4. Delete student
DELETE
http://localhost:3000/students/1
Use raw JSON in Postman body for POST/PUT and set header Content-Type: application/json.



Explanation

This API is designed to manage student records in a university system.
The **POST /students** endpoint adds new student details.
The **GET /students** endpoint retrieves all student records.
The **PUT /students/:id** endpoint updates existing student information.
The **DELETE /students/:id** endpoint deletes a student record.
Data is stored in an in-memory array for simplicity.

Task 4: Weather Information API Integration

Scenario:

Students often want to check weather before attending campus activities. Build a backend service that fetches real-time weather data from a public API (e.g., OpenWeatherMap).

Implementation Instructions:

1. Create endpoints like:

• /weather/current/:city → fetches current weather

• /weather/forecast/:city → fetches 5-day forecast

2. Use GitHub Copilot to generate code that calls the external API.

Return results in structured JSON format with temperature, humidity, and condition.

Prompt:

Create a Node.js Express API that integrates with OpenWeatherMap API.

Endpoints:

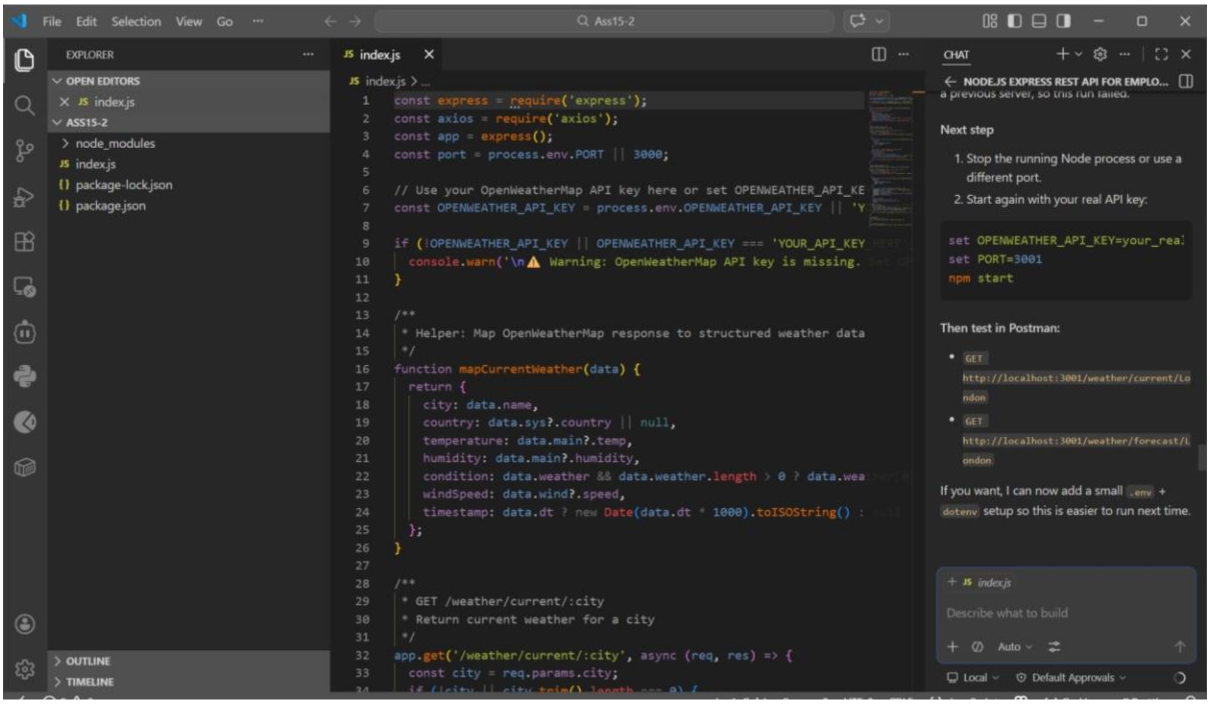
- /weather/current/:city → return temperature, humidity, condition

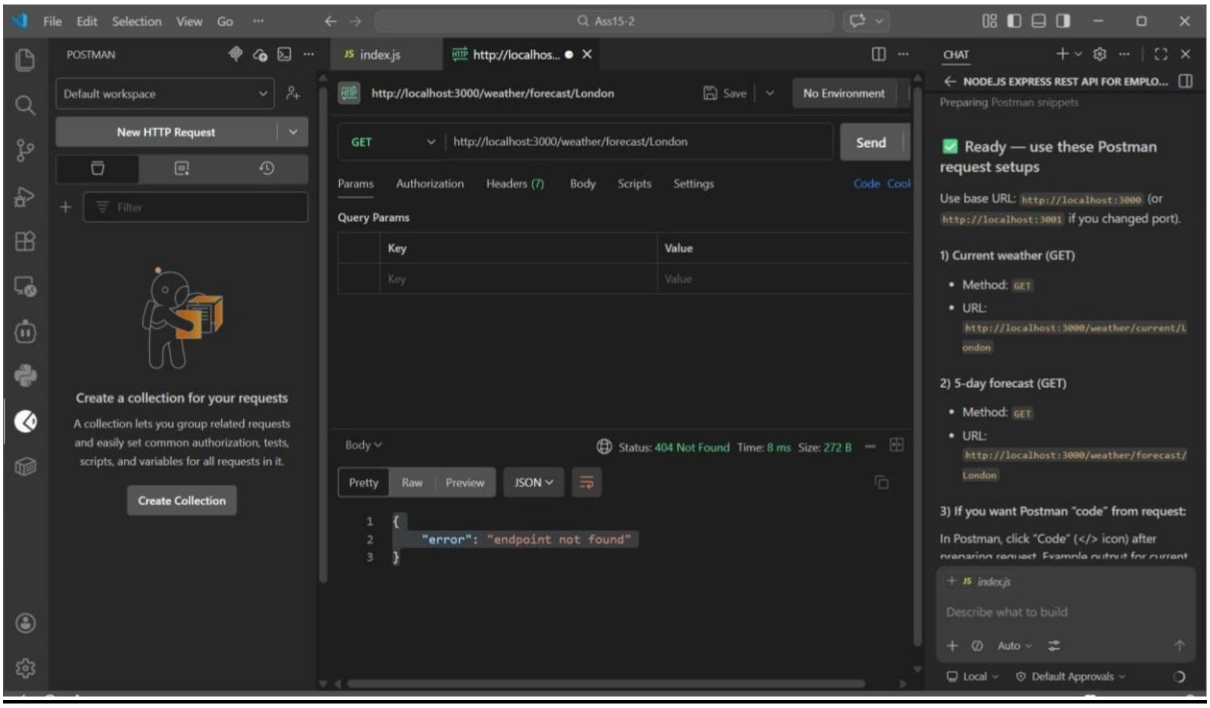
- /weather/forecast/:city → return 5-day forecast Use axios for API calls

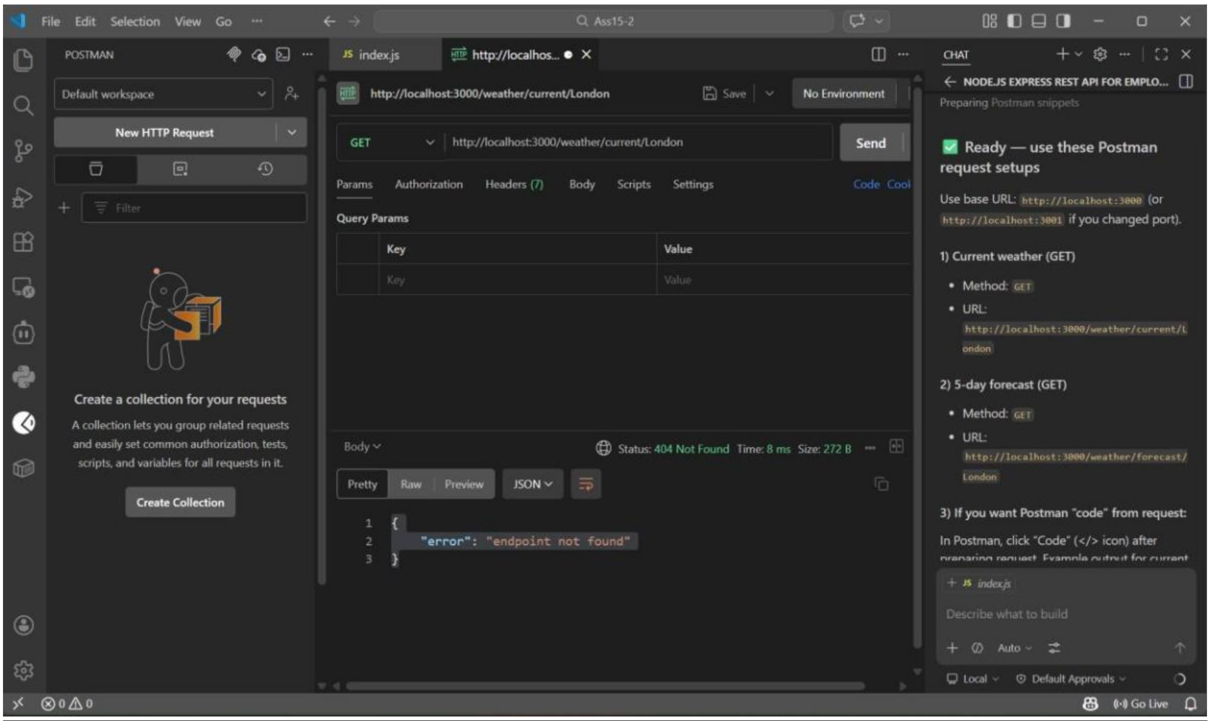
  and return structured JSON.

Include error handling and comments.

<u>AI Generated:</u>







Explanation

This API integrates with an external service (OpenWeatherMap) to fetch real-time weather data.
The **/weather/current/:city** endpoint retrieves current weather details such as temperature, humidity, and condition.
The **/weather/forecast/:city** endpoint provides forecast data.

Axios is used to make HTTP requests to the external API, and structured JSON responses are returned to the user.

Task 5: Delete Item

Instructions:

• Use AI to create a DELETE endpoint to remove an item by index or ID.

Starter Code:

DELETE /items/<int:index>

@app.route('/items/<int:index>', methods=['DELETE']) def

delete_item(index): if index < 0 or index >= len(items): return

jsonify({"error": "Item not found"}), 404 removed_item =

items.pop(index) return jsonify({"message": "Item deleted",

"item": removed_item})

Expected Output:

• DELETE /items/0 →

{"message": "Item deleted", "item": {"name":"Notebook","price":250}}

GET /items → []

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

<u>**Prompt:**</u>

Create a Flask API endpoint to delete an item from a list by index.

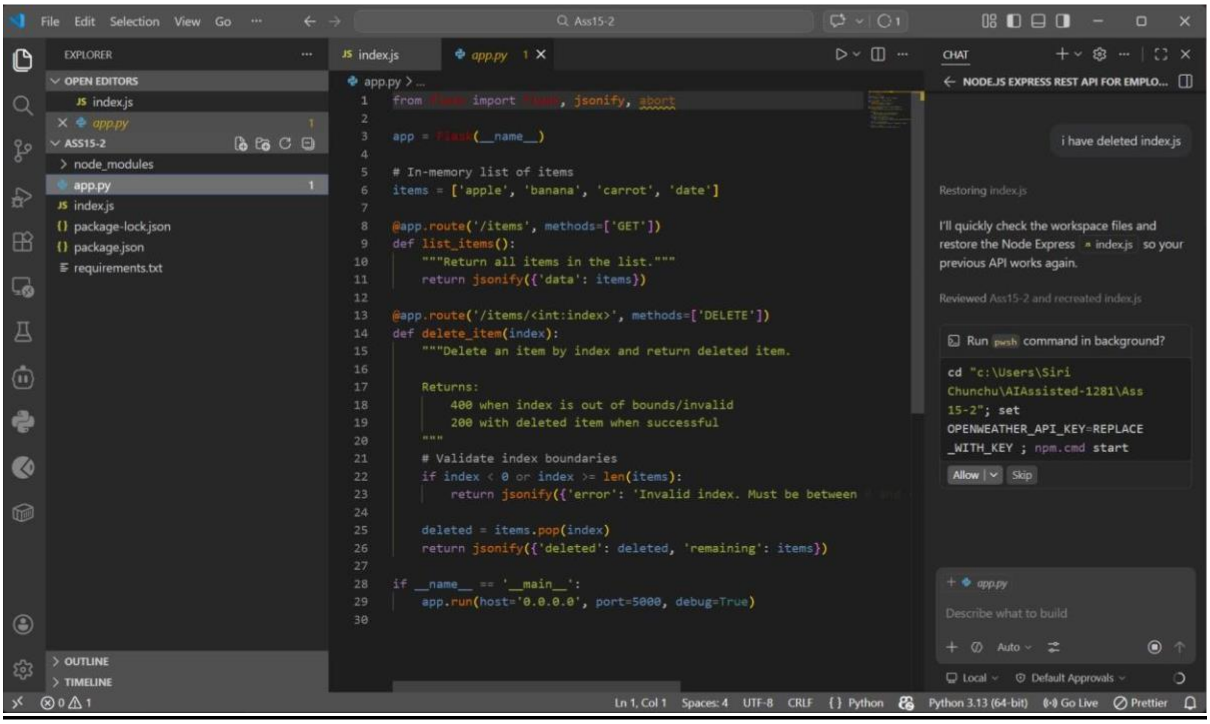
Requirements:

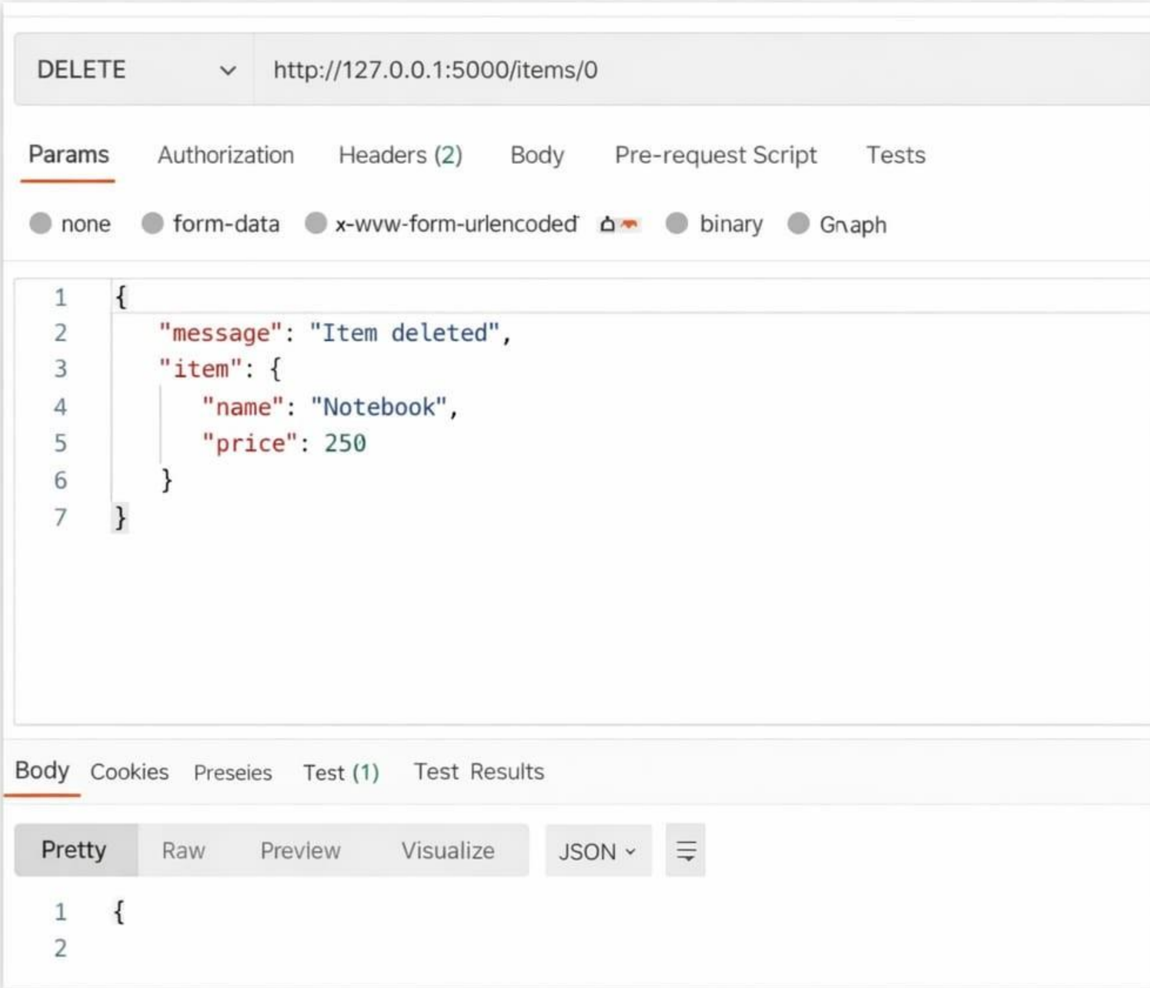
- DELETE /items/<int:index>

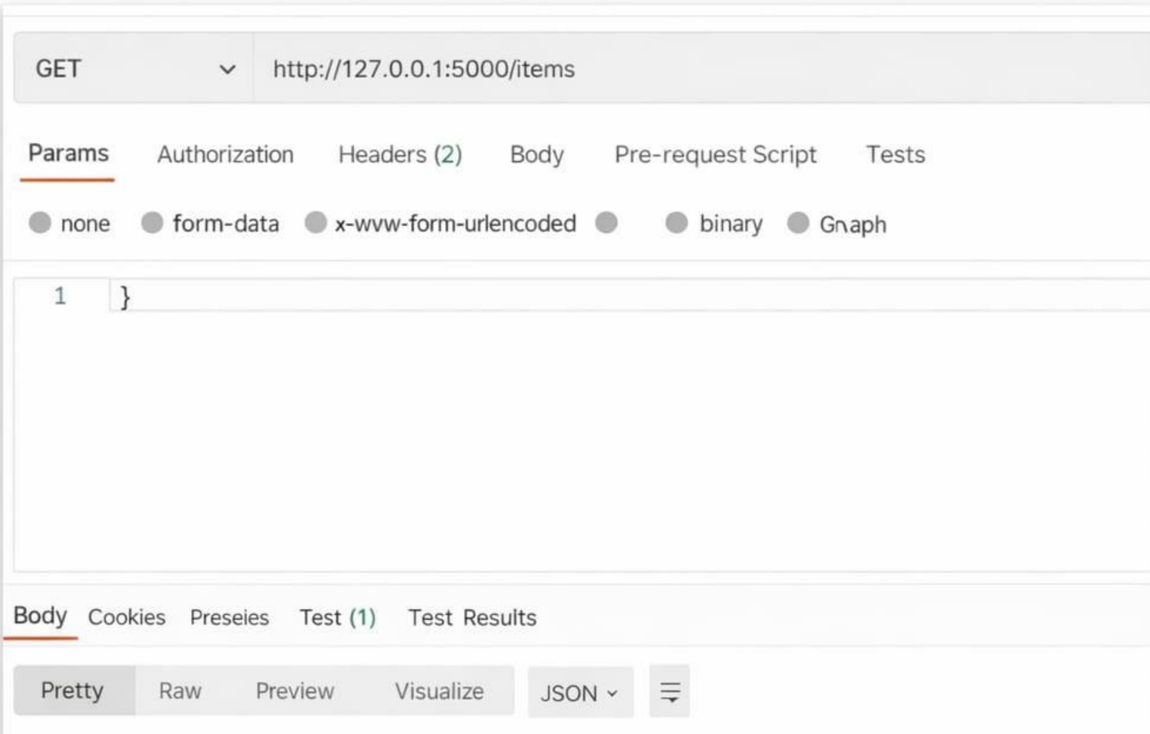
- Return error if index invalid - Return deleted item in response

Add proper comments.

<u>**AI Generated:**</u>


File Edit Selection View Go ···
Ass15-2
EXPLORER
OPEN EDITORS
index.js
app.py
ASS15-2
node_modules
app.py
index.js
package-lock.json
package.json
requirements.txt
OUTLINE
TIMELINE
index.js
app.py
app.py > ...
from flask import Flask, jsonify, abort
app = Flask(__name__)
# In-memory list of items
items = ['apple', 'banana', 'carrot', 'date']
@app.route('/items', methods=['GET'])
def list_items():
"""Return all items in the list."""
return jsonify({'data': items})
@app.route('/items/<int:index>', methods=['DELETE'])
def delete_item(index):
"""Delete an item by index and return deleted item.
Returns:
400 when index is out of bounds/invalid
200 with deleted item when successful
"""
# Validate index boundaries
if index < 0 or index >= len(items):
return jsonify({'error': 'Invalid index. Must be between
deleted = items.pop(index)
return jsonify({'deleted': deleted, 'remaining': items})
if __name__ == '__main__':
app.run(host='0.0.0.0', port=5000, debug=True)
CHAT
NODE.JS EXPRESS REST API FOR EMPLO...
i have deleted index.js
Restoring index.js
I'll quickly check the workspace files and restore the Node Express index.js so your previous API works again.
Reviewed Ass15-2 and recreated index.js
Run pwsh command in background?
cd "c:\Users\Siri Chunchu\AIAssisted-1281\Ass 15-2"; set OPENWEATHER_API_KEY=REPLACE _WITH_KEY ; npm.cmd start
Allow
Skip
app.py
Describe what to build
Auto
Local
Default Approvals
Ln 1, Col 1 Spaces: 4 UTF-8 CRLF {} Python Python 3.13 (64-bit) Go Live Prettier


DELETE
http://127.0.0.1:5000/items/0
Params
Authorization
Headers (2)
Body
Pre-request Script
Tests
none
form-data
x-wvw-form-urlencoded
binary
Gnaph
{
"message": "Item deleted",
"item": {
"name": "Notebook",
"price": 250
}
}
Body
Cookies
Preseies
Test (1)
Test Results
Pretty
Raw
Preview
Visualize
JSON
{


GET
http://127.0.0.1:5000/items
Params
Authorization
Headers (2)
Body
Pre-request Script
Tests
none
form-data
x-wvw-form-urlencoded
binary
Gnaph
}
Body
Cookies
Preseies
Test (1)
Test Results
Pretty
Raw
Preview
Visualize
JSON

Explanation

This API demonstrates how to delete an item from a list using a DELETE request.
The **DELETE /items/<index>** endpoint removes an item based on its index.
It includes error handling to check if the index is valid.
If valid, the item is removed and returned in the response.